```
##WARDA ASIF
BATCH MAY-JULY



###Itroduction:
Predicting houses prices in BostonHousing.csv
```

```
##Import Libraries
```

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
## Uploading the Data
```

```
from google.colab import files

uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving BostonHousing.csv to BostonHousing (1).csv

```
import pandas as pd

df = pd.read_csv('BostonHousing.csv')
df.head()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396

```
##Exploratory Data Analysis
```

```
#EDA
```

```
df.shape
```

```
(506, 14)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   crim     506 non-null    float64
 1   zn       506 non-null    float64
 2   indus    506 non-null    float64
 3   chas     506 non-null    int64  
 4   nox      506 non-null    float64
 5   rm       501 non-null    float64
 6   age      506 non-null    float64
 7   dis      506 non-null    float64
 8   rad      506 non-null    int64  
 9   tax      506 non-null    int64  
 10  ptratio  506 non-null    float64
 11  b        506 non-null    float64
 12  lstat    506 non-null    float64
 13  medv     506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
```

```
df.describe()
```

	crim	zn	indus	chas	nox	rm	
count	506.000000	506.000000	506.000000	506.000000	506.000000	501.000000	5
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284341	
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.705587	
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	
25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.884000	
50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208000	
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.625000	
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	1

```
df.isnull().sum()
```

	0
crim	0
zn	0
indus	0
chas	0
nox	0
rm	5
age	0
dis	0
rad	0
tax	0
ptratio	0
b	0
lstat	0
medv	0

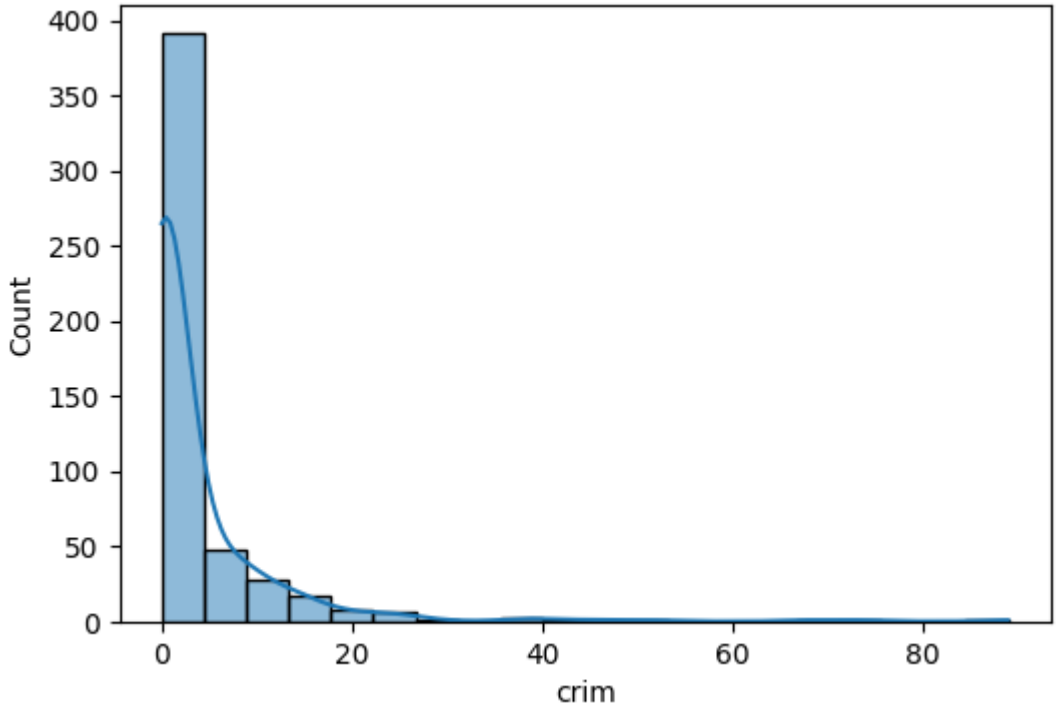
dtype: int64

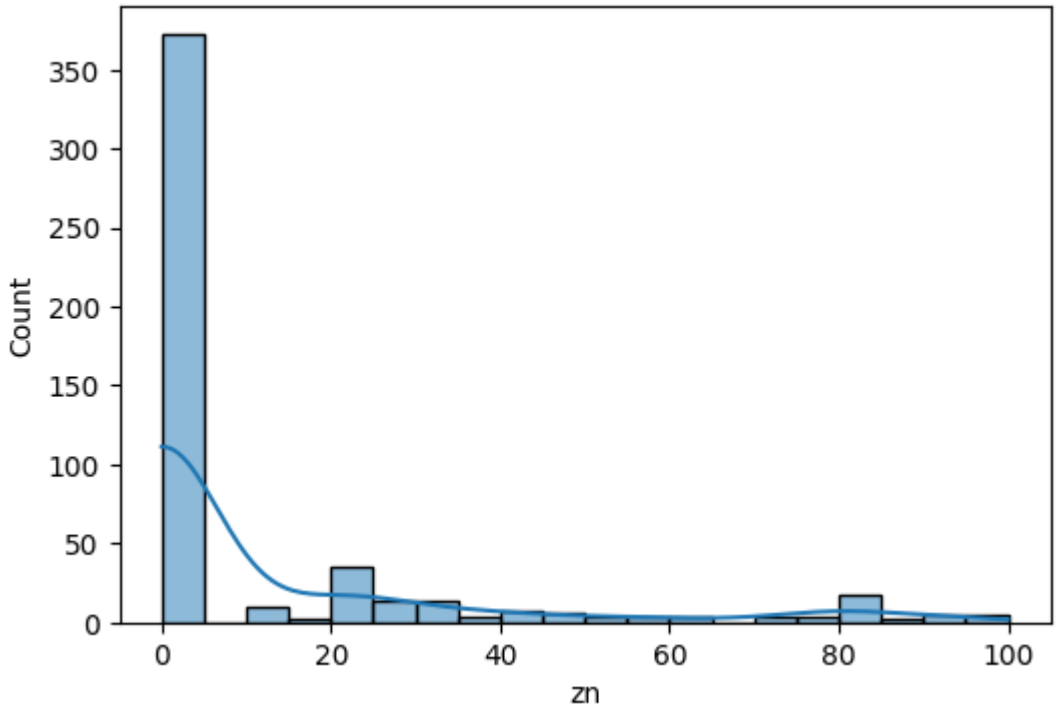
```
df.columns
```

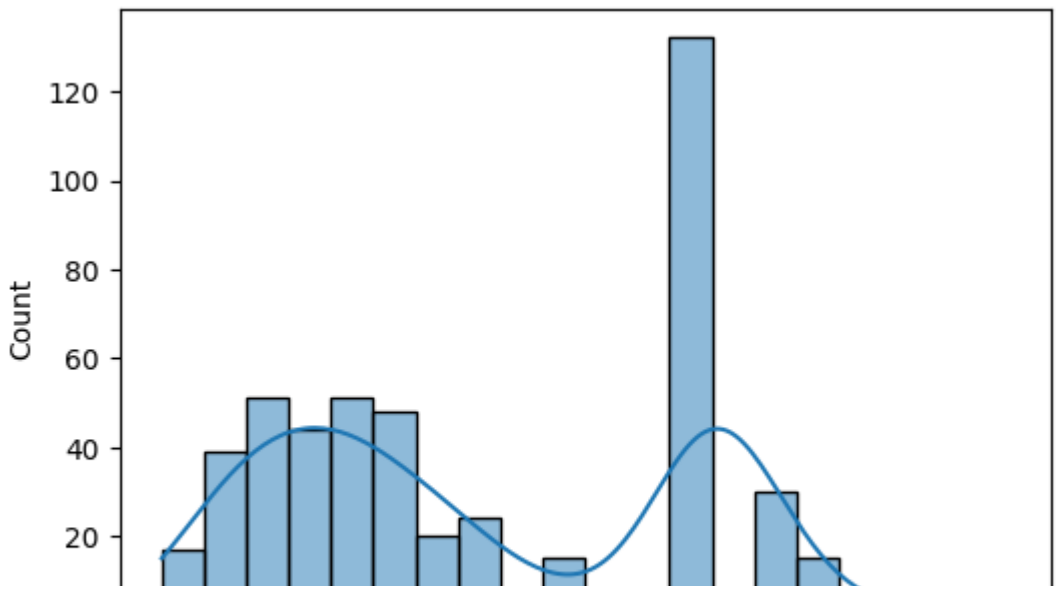
```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad',
'tax',
       'ptratio', 'b', 'lstat', 'medv'],
      dtype='object')
```

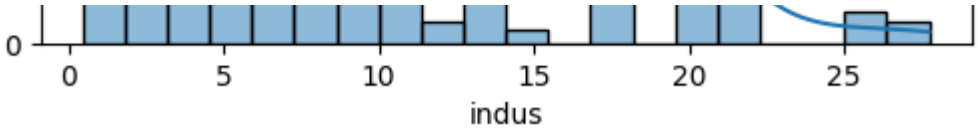
```
import matplotlib.pyplot as plt
import seaborn as sns
```

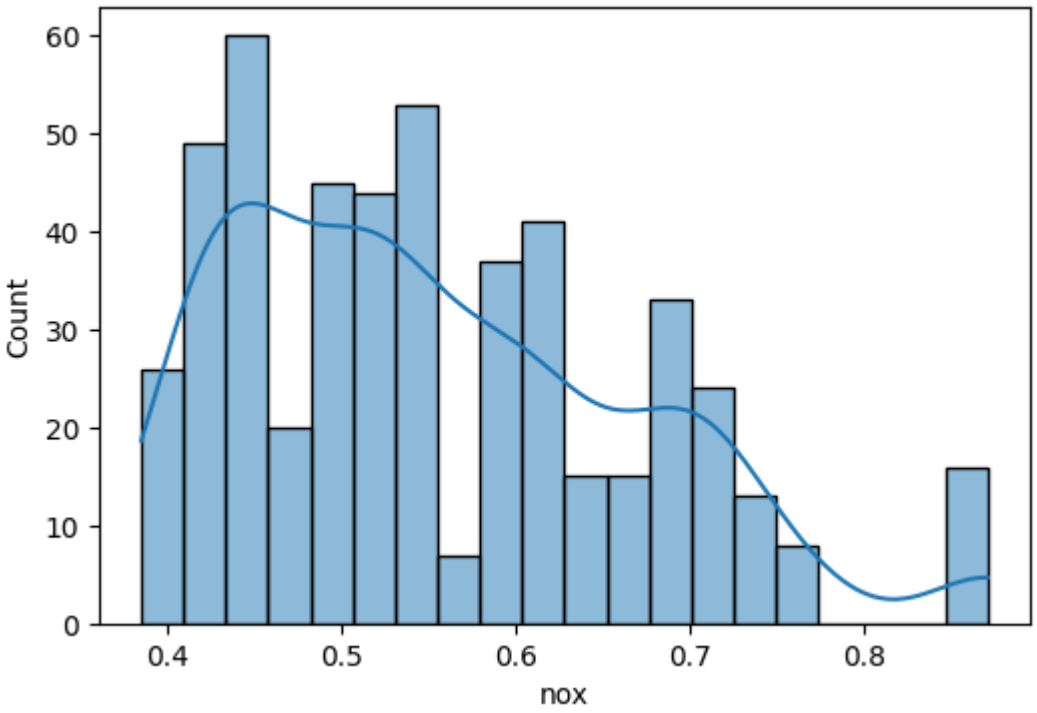
```
numeric_columns = [
    "crim", "zn", "indus", "nox", "rm",
    "age", "dis", "rad", "tax",
    "ptratio", "b", "lstat", "medv"
]

for col in numeric_columns:
    plt.figure(figsize=(6, 4))
    sns.histplot(df[col], kde=True, bins=20)
```

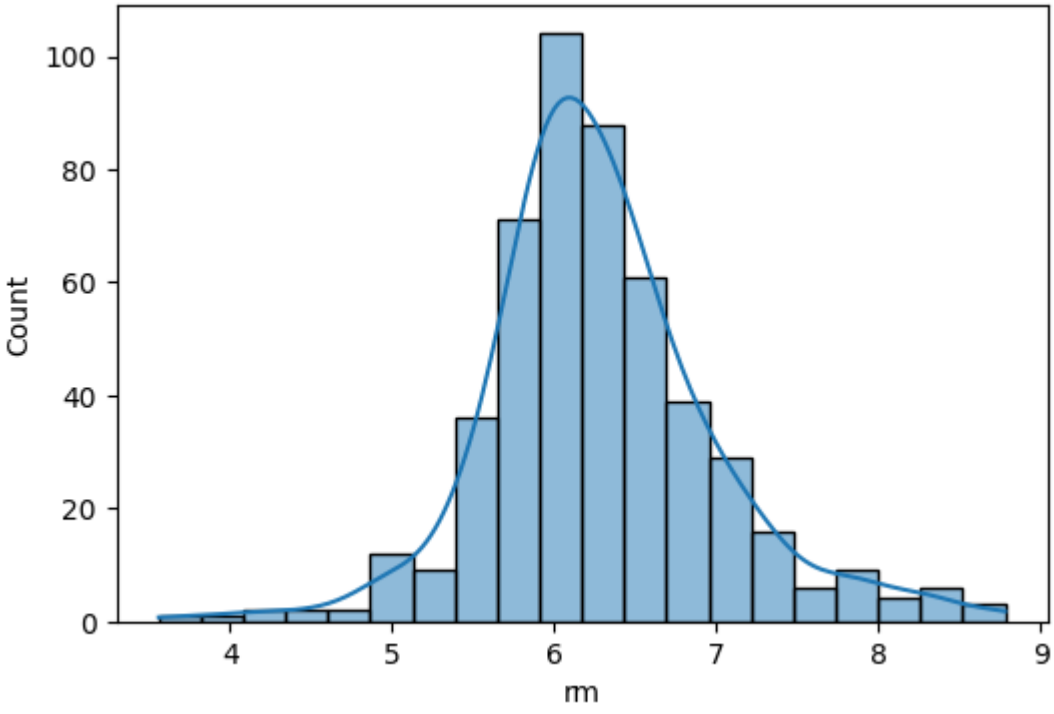

Count
crim


Count
zn

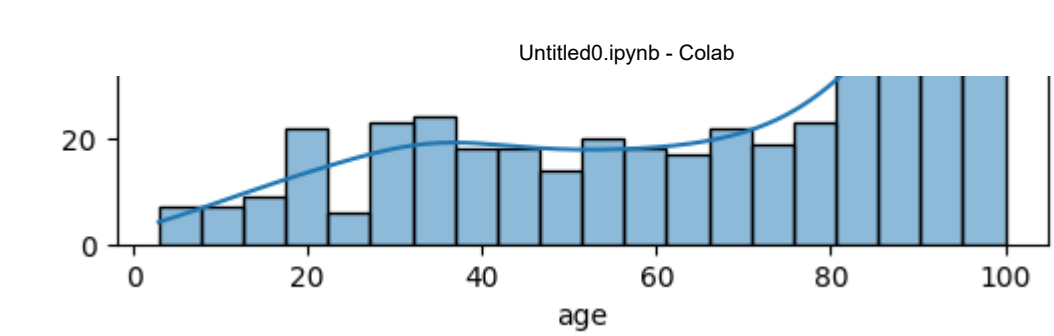

Count

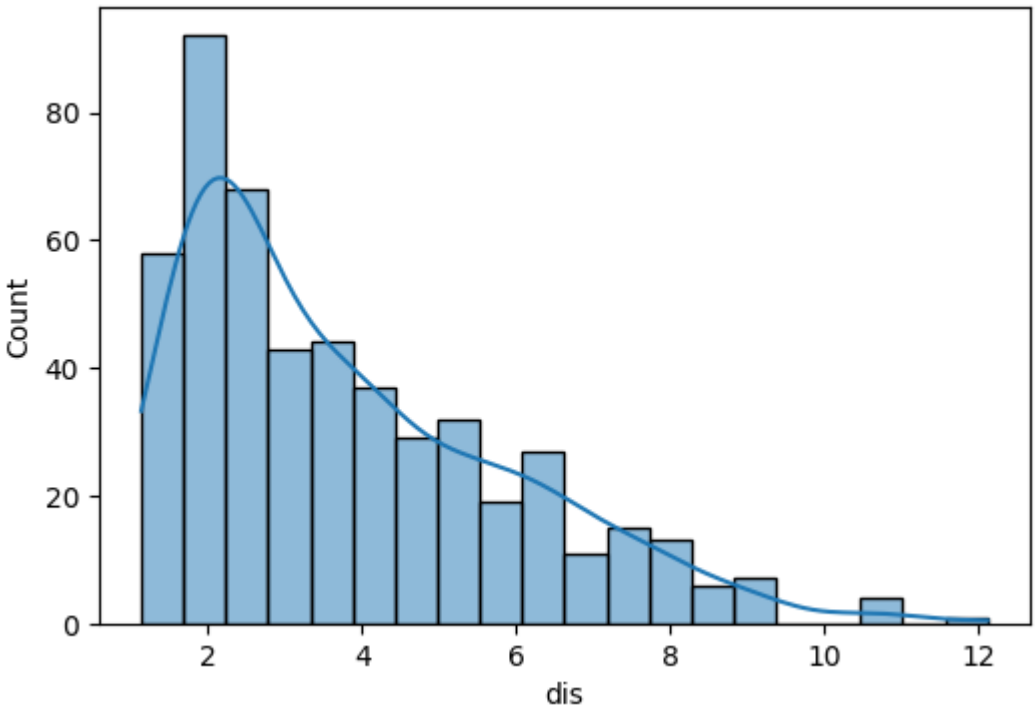

0
0
5
10
15
20
25
indus

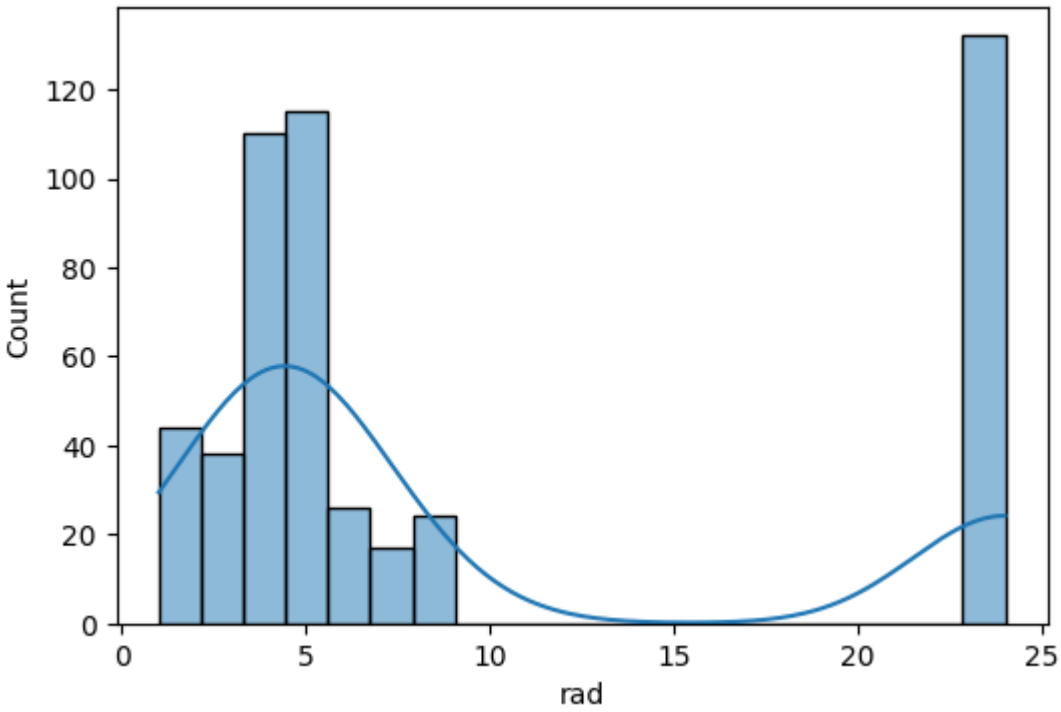

60
50
40
30
20
10
0
Count
0.4
0.5
0.6
0.7
0.8
nox

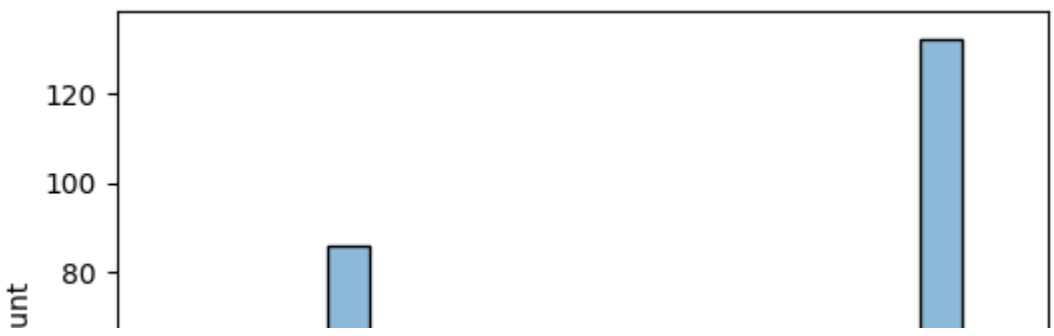

100
80
60
40
20
0
Count
4
5
6
7
8
9
rm

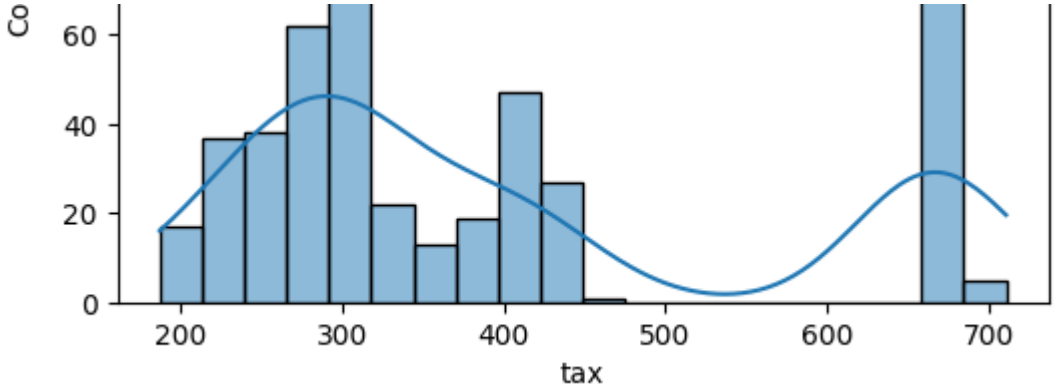

100
80
60
40
Count

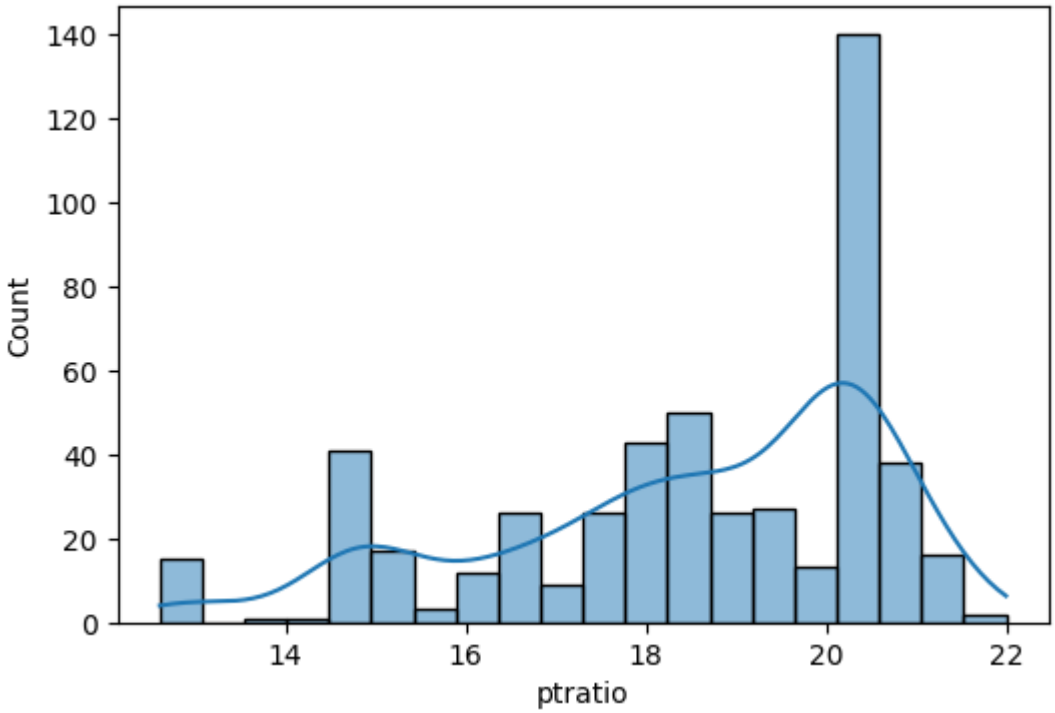

20
0
0
20
40
60
80
100
age

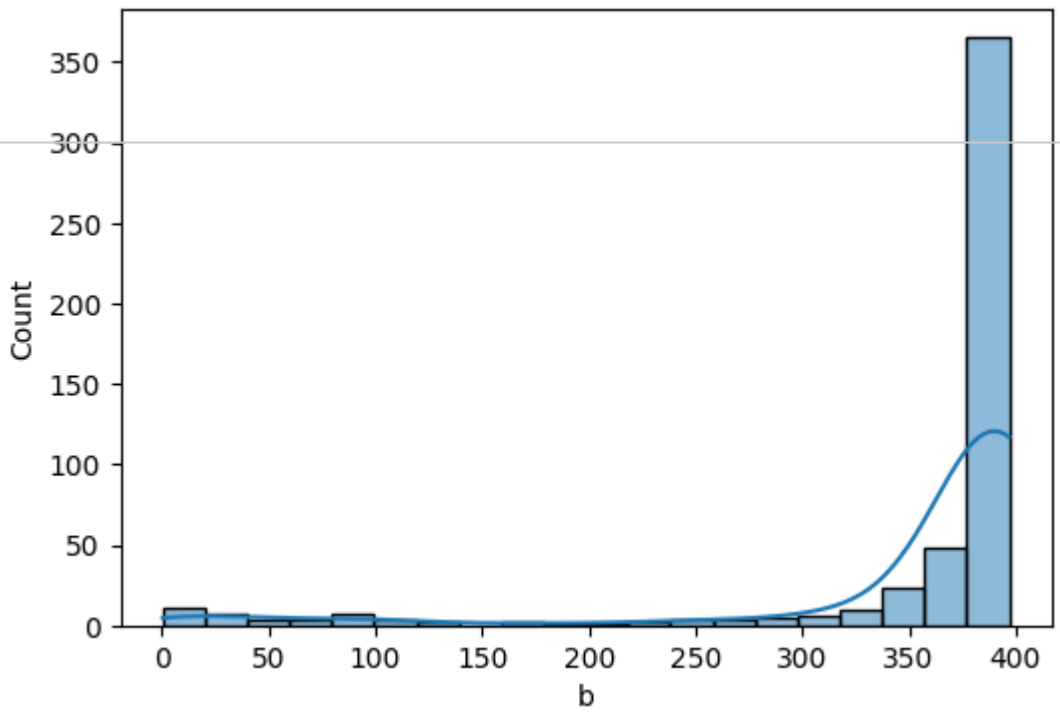

80
60
40
20
0
Count
2
4
6
8
10
12
dis


120
100
80
60
40
20
0
Count
0
5
10
15
20
25
rad


120
100
80
unt

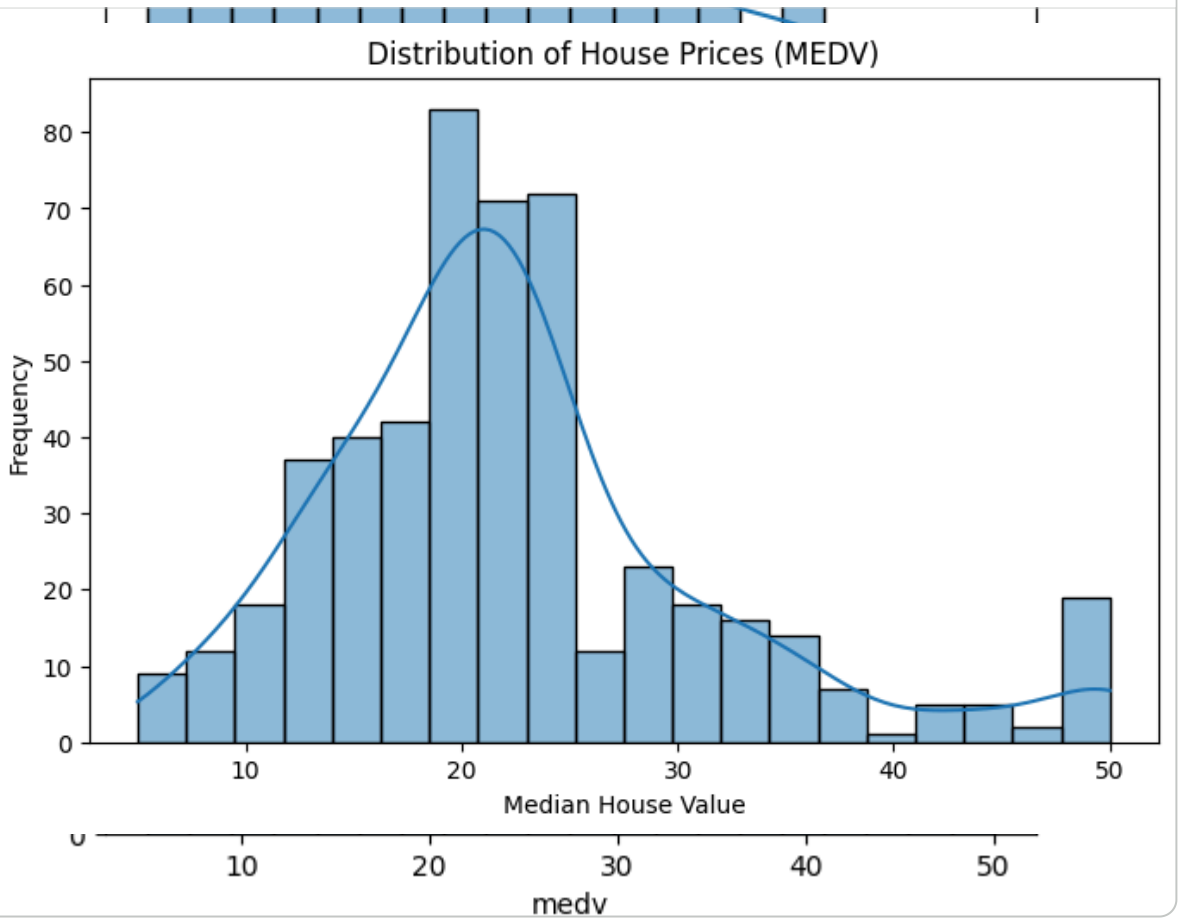

Co
60
40
20
0
200
300
400
500
600
700
tax


140
120
100
80
60
40
20
0
Count
14
16
18
20
22
ptratio


350
300
250
200
150
100
50
0
Count
0
50
100
150
200
250
300
350
400
b

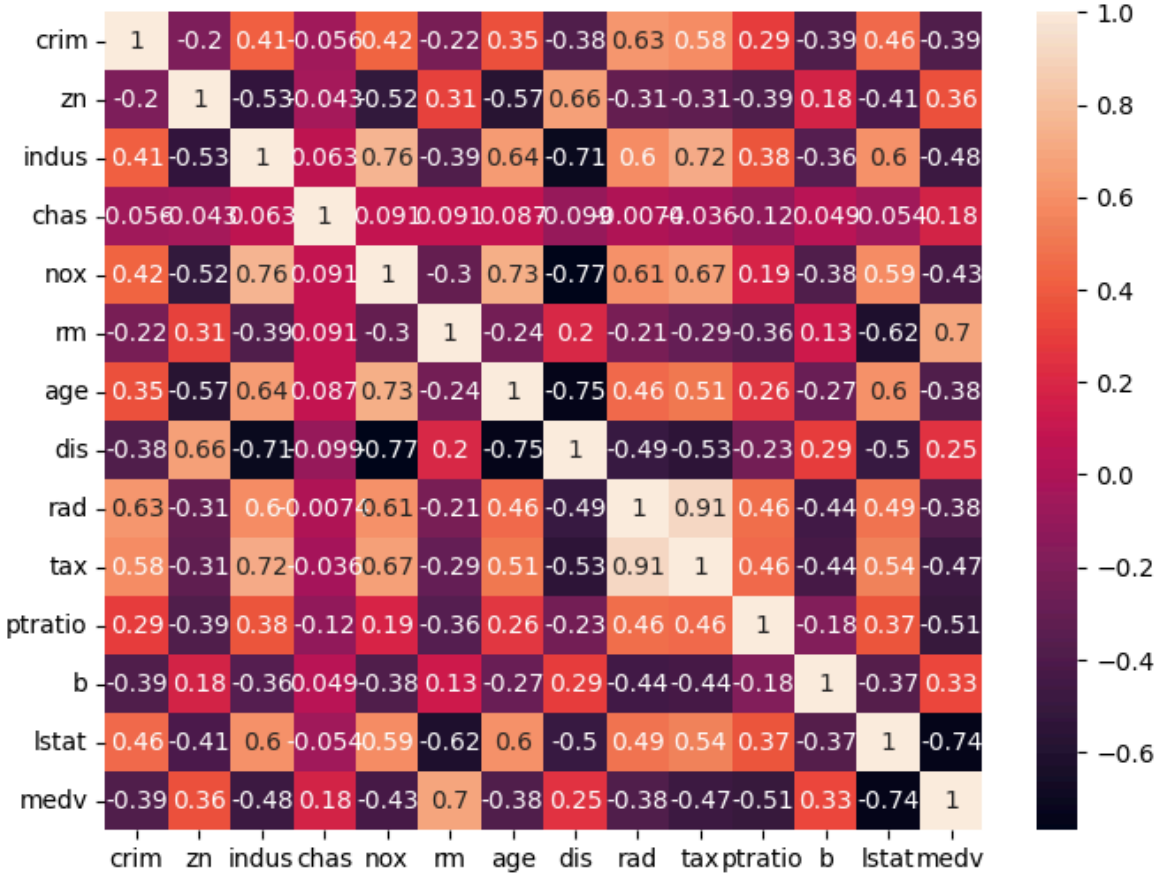

60
50

```
plt.figure(figsize=(8,5))
sns.histplot(df["medv"], kde=True, bins=20)

plt.title("Distribution of House Prices (MEDV)")
plt.xlabel("Median House Value")
plt.ylabel("Frequency")

plt.show()
```




```
plt.figure(figsize = (8,6))
sns.heatmap(df.corr(numeric_only = True),annot = True)
```

```
<Axes: >
```




```
##Data processing
```

```
# Features (Independent Variables)
X = df.drop('medv', axis=1)

# Target (Dependent Variable)
y = df['medv']

print("Features shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features shape: (506, 13)
Target shape: (506,)
```

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='median')

X = df.drop('medv', axis=1)
y = df['medv']

X = imputer.fit_transform(X)
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

##Train the Model

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")

X = imputer.fit_transform(X)
```

```
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_scor
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R² Score:", r2)
```

```
MAE : 3.2114487155504143
MSE : 24.456593109406292
RMSE: 4.945360766355301
R² Score: 0.6665030487253184
```

prediction Results

```
results = pd.DataFrame({
    "Actual Price": y_test,
```